

React Rendering Rules – Complete Detailed Explanation

This section contains **all React rendering cases** you asked for, explained clearly and technically, covering re-renders, remounts, props, functions, useCallback, and React.memo behavior.

★ CASE 1 — Child component used inside parent with *no props*

```
function Parent() {  
  return <Child />;  
}
```

Will Child re-render if Parent re-renders?

🐱 **YES — always**

React re-renders all children by default.

Even though no props are passed, the `<Child />` function is **called again**.

How to prevent it?

```
const Child = React.memo(function Child() {...});
```

React.memo stops re-render because props are empty and identical.

★ CASE 2 — Parent passes props, but props **DO NOT** change

```
function Parent() {  
  const [count, setCount] = useState(0);  
  return <Child value={10} />;  
}
```

Will Child re-render?

🐼 **YES — without memo**

React re-renders Child every time Parent renders.

With React.memo

```
const Child = React.memo(function Child({ value }) {...});
```

Now React checks:

```
old props = { value: 10 }  
new props = { value: 10 }
```

Identical → Child does NOT re-render.

★ CASE 3 — Passing a function as prop (without useCallback)

```
function Parent() {  
  const handleClick = () => {};  
  return <Child onClick={handleClick} />;  
}
```

Will Child re-render?

🐼 **YES — always**

Because the function prop is recreated on every parent render.

React.memo sees:

```
old function ref: A  
new function ref: B  
A !== B → re-render
```

How to stop this?

Use useCallback + React.memo

```
const handleClick = useCallback(() => {}, []);  
const Child = React.memo(function Child({ onClick }) {...});
```

Now no re-render occurs.

★ CASE 4 — Using useCallback WITHOUT React.memo

```
const fn = useCallback(() => {}, []);  
return <Child onClick={fn} />;
```

Will Child re-render?

🐱 YES

Because React.memo is NOT used, `useCallback` **only stabilizes the function reference**, it DOES NOT stop rendering.

★ CASE 5 — BEST CASE: React.memo + useCallback

```
const handleClick = useCallback(() => {}, []);  
const Child = React.memo(function Child({ onClick }) {...});
```

This combination: ✓ Parent re-renders ✓ Function reference stays same ✓ Props unchanged ✓
React.memo skips re-render

This is the **correct optimization pattern**.

★ FINAL SUMMARY TABLE

Scenario	Parent Re-renders (WITH memo)	Child Re-renders (NO memo)	Child Re-renders
No props passed	Yes	Yes	No
Props unchanged	Yes	Yes	No
Function passed	Yes	Yes	Yes
Function + useCallback	Yes	Yes	No (with memo)
useCallback only	Yes	Yes	Yes

★ Absolute Rules to Remember

- `useCallback` **does NOT stop rendering.**
- `React.memo` **prevents rerender** by comparing props.
- For function props → `useCallback` + `React.memo` **together** are required.
- Child without props STILL re-renders unless wrapped in `React.memo`.

End of React Rendering Rules Report.